

Carrier Projection Layer for the Δ _C-Orbit Frame

Platform-Agnostic Barcode and Visual Transport Model

Status: Engineering Draft

Scope: Barcode carriers, visual transport, physical projection, frame preservation

Depends on: `omi.delta_orbital_lexer.v0`

Canonical payload: `S = uint16_t[8]`

Canonical validity predicate: `Q(S) = 0`

1. Purpose

This document defines barcode and visual symbol systems as carrier projections for a fixed-width algebraic frame.

The canonical frame is:

```
S = uint16_t[8]
```

with layout:

```
0xLL00-0x03bf-0xNNNN-0x2bLL-0x2fLL-0xMMMM-0x039f-0xLLff
```

Barcode variants are not separate computational systems. They are transport surfaces for the same frame.

The carrier layer is responsible for preserving and recovering `S`.

It is not responsible for validating `S`.

Validation remains the responsibility of the algebraic validator:

```
Q(S) = 0
```

2. Carrier Abstraction

A carrier implements two functions:

```
encode(S) → scannable representation  
decode(scannable representation) → S or error
```

The required preservation invariant is:

```
decode(encode(S)) = S
```

for every supported frame `S` within the carrier's capacity and error-correction limits.

A carrier may provide checksums, error correction, orientation marks, finder patterns, color coding, or redundancy. These mechanisms improve transport reliability but do not replace algebraic validation.

The validation pipeline is:

```
carrier decode  
  ↓  
S = uint16_t[8]  
  ↓  
Q(S) = 0  
  ↓  
extract LL, NN, MM  
  ↓  
truth-row resolution  
  ↓  
receipt / replay ring
```

3. Separation of Carrier and Authority

The carrier is transport.

The algebraic frame is authority.

This distinction is mandatory.

A barcode may successfully scan and still produce an invalid frame. Such a frame must be rejected by `Q(S)`.

Likewise, a carrier checksum may pass while the algebraic frame fails. Carrier-level integrity only proves that the visual or physical symbol was decoded according to that carrier's rules. It does not prove that the decoded frame is structurally valid under the ABI.

The canonical rule is:

No carrier validates authority.
Only $Q(S)$ validates structural authority.

4. Carrier Pipeline

A full carrier pipeline has four stages:

1. Physical or visual acquisition
2. Carrier decoding
3. Algebraic frame validation
4. Truth-row resolution and replay receipt

Expanded:

```
scan / receive / read
  ↓
carrier decoder
  ↓
recover  $S[0..7]$ 
  ↓
verify  $Q(S) = 0$ 
  ↓
extract LL, NN, MM
  ↓
fanoTruthResolver(LL, NN, MM)
  ↓
steps  $\in$  valid bound
  ↓
CAS cursor advance
  ↓
write 64-bit receipt
```

5. Linear Barcode Carriers

Linear carriers encode the frame as a one-dimensional sequence of bars and spaces.

They are useful for streams, labels, tapes, receipts, low-cost scanners, and industrial workflows.

Representative linear carriers:

Carrier	Transport Role	Engineering Use
ITF	pulse / metric synchronization	physical-layer frame timing
Codabar	raw segment tape	simple segment transport
Code128	compact checked carrier	dense 1D frame encoding
Code16K	multi-row carrier	larger frame blocks or redundancy
Code93	checked witness carrier	higher-redundancy 1D witness

A linear carrier may transport:

```
full S vector
compressed S vector
frame pointer
checkpoint
receipt
redundant witness
```

depending on carrier capacity.

The ABI does not require every carrier to store the full 128-bit frame directly. It only requires that any recovered frame be converted into canonical `uint16_t[8]` form before validation.

6. Linear Precision Ladder

The linear carriers may be organized as a precision ladder:

```
binary16
  ↓
binary128
  ↓
binary192
  ↓
binary256
```

Interpretation:

```
binary16  synchronization, short tokens, partial checkpoints
binary128  full canonical frame
```

```
binary192  frame plus additional row/redundancy data
binary256  frame plus witness, checksum, or error-correction material
```

This ladder should be treated as an implementation profile, not as a separate semantic layer.

The frame remains:

```
S = uint16_t[8]
```

The validator remains:

```
Q(S) = 0
```

7. Matrix Barcode Carriers

Matrix carriers encode the frame on a two-dimensional surface.

They are useful for phone scanning, web interfaces, printed artifacts, physical signage, device pairing, and high-density transport.

Representative matrix carriers:

Carrier	Assigned Plane	Transport Role
Aztec	FS	orientation and world-frame projection
MaxiCode	GS	grouped or batch transport
JABCode	RS	high-capacity record transport
BeeTag	US	short visual pointer or unit tag

The plane assignments are engineering conventions:

```
FS  frame / file-scale orientation
GS  group-scale batch transport
RS  record-scale storage
US  unit-scale pointer
```

These assignments do not change the algebraic frame. They describe how different visual carriers may be used in a layered transport system.

8. Matrix Carrier Responsibilities

A matrix carrier may provide:

```
finder pattern  
orientation detection  
grid recovery  
error correction  
color-channel capacity  
batch grouping  
short-link reference
```

But after decoding, the result must still be reduced to either:

```
S = uint16_t[8]
```

or:

```
a pointer that resolves to S
```

No matrix carrier may bypass $Q(S)$.

9. Stream Carrier Role

ITF or a similar pulse-oriented carrier may be used as a stream synchronization layer.

In this role, it does not necessarily carry the complete frame in one symbol. Instead, it can provide:

```
start / stop boundary  
pulse timing  
clock recovery  
segment delimiter detection  
lightweight binary checkpoint
```

A stream-oriented boot pipeline may look like:

```
ITF pulse train  
↓  
frame synchronization  
↓
```

```

segment extraction
  ↓
assembly into S[0..7]
  ↓
Q(S) validation

```

The stream carrier is closest to a physical-layer role.

10. Unified Transport Model

All carriers participate in the same transport contract:

```

carrier representation
  ↓
decode
  ↓
canonical S vector
  ↓
algebraic validator

```

Carrier-specific error correction is allowed. Carrier-specific encoding density is allowed. Carrier-specific visual geometry is allowed.

Carrier-specific meaning is not allowed.

Meaning enters only after:

$$Q(S) = 0$$

and truth-row resolution succeeds.

11. Platform-Agnostic Implementation

This model is platform-agnostic because the computational authority is not tied to any barcode library or scanner.

The stable pieces are:

Component	Platform-Agnostic Basis
frame	<code>uint16_t[8]</code>

Component	Platform-Agnostic Basis
validation	integer arithmetic
resolver	16-bit rotations and XOR
receipt	64-bit packing
cursor	atomic compare-exchange where available
carrier	replaceable encoder / decoder

The same canonical frame can be:

```

printed as a barcode
displayed as a matrix tag
transmitted as a pulse stream
stored in a database
embedded in a web page
sent over a local network
compiled into WASM memory

```

The transport changes. The frame does not.

12. Example End-to-End Barcode Flow

Example using a visual carrier:

```

Aztec symbol
  ↓
scanner decodes bytes
  ↓
bytes map to S[0..7]
  ↓
Q(S) = 0
  ↓
LL, NN, MM extracted
  ↓
fanoTruthResolver(LL, NN, MM)
  ↓
steps returned
  ↓
receipt packed

```


↓
cursor advanced atomically

Example using a pointer carrier:

BeeTag
↓
scanner resolves short pointer
↓
pointer retrieves stored frame S
↓
 $Q(S) = 0$
↓
truth-row resolution
↓
receipt

Example using a stream carrier:

ITF pulse stream
↓
clock recovery
↓
segment reconstruction
↓
 $S[0..7]$
↓
 $Q(S) = 0$
↓
resolver
↓
receipt

13. Security Properties

The carrier layer provides useful security boundaries.

13.1 Carrier Success Is Not Frame Validity

A successful scan does not imply a valid frame.

Only $Q(S) = 0$ implies structural validity.

13.2 Carrier Corruption Is Caught Twice

Corruption may be caught by:

carrier checksum / ECC

and then again by:

$Q(S)$

These are independent checks.

13.3 Visual Spoofing Does Not Create Authority

A symbol that looks valid must still decode into the exact canonical segment vector.

13.4 Carrier Choice Does Not Change Meaning

The same frame encoded in two different carriers has the same authority if both decode to the same S .

13.5 Invalid Frame Pointers Do Not Resolve

Pointer-style carriers must ultimately resolve to a valid S . Otherwise the pipeline rejects.

14. Conformance Requirements

A conforming carrier profile should define:

1. How S is serialized into carrier bytes
2. How carrier bytes are recovered into S
3. Endianness rules
4. Error-correction behavior
5. Maximum payload capacity
6. Whether the carrier stores S directly or stores a pointer to S
7. Required golden round-trip tests

Minimum round-trip requirement:

$\text{decode}(\text{encode}(S)) = S$

for every supported golden frame.

Minimum validation requirement:

```
decode(symbol) = S  
verifyOrbitLexer(S) = Q(S)
```

No carrier profile may replace `verifyOrbitLexer`.

15. Engineering Summary

The barcode layer is a carrier projection layer.

It does not define computation.

It transports the canonical frame.

The locked statement is:

```
Barcode variants are not separate encoding systems.  
They are different transport surfaces for the same algebraic frame S =  
uint16_t[8].  
Linear carriers provide one-dimensional precision and stream profiles.  
Matrix carriers provide two-dimensional geometric and visual profiles.  
Pointer carriers may reference stored frames.  
All carriers must reduce to the same canonical segment vector.  
Validation happens after decoding through  $Q(S) = 0$ .  
The bounded resolver and replay ring are carrier-agnostic.
```

16. Minimal Canonical Statement

```
The carrier preserves S.  
The validator proves S.  
The resolver interprets LL, NN, MM.  
The ring records the accepted row.
```

Or:

```
transport the frame,  
prove the frame,
```

```
resolve the row,  
record the receipt.
```